

Reflexion - Evidencia 1

Cesar Cardenas | A00844803

Importancia de ordenar y buscar logs

Los logs registran eventos de distintos equipos y no siempre llegan en orden. Ordenarlos permite reconstruir lo que ocurrio, encontrar fallas y revisar incidentes de seguridad. Si los registros permanecen desordenados, una busqueda por fechas puede omitir eventos o presentar una secuencia equivocada.

Despues de ordenar, la busqueda binaria permite localizar los limites de un rango sin revisar uno por uno los 6818 registros. Cada limite requiere $O(\log n)$. Esta diferencia se vuelve mas importante cuando los archivos crecen.

Comparacion de algoritmos

Swap Sort, Selection Sort y Bubble Sort son faciles de entender, pero realizan una cantidad cuadratica de comparaciones. Selection Sort busca el menor elemento y lo coloca en cada posicion. Bubble Sort compara vecinos y mueve los mayores hacia el final. En mi version, Bubble Sort no tiene una salida anticipada, por lo que incluso su mejor caso permanece en $O(n^2)$.

Insertion Sort coloca cada registro en la parte ya ordenada. Su mejor caso es $O(n)$, por lo que puede beneficiarse bastante del archivo casi ordenado. Merge Sort divide el vector y vuelve a unir las partes ordenadas; mantiene $O(n \log n)$ tanto en el mejor como en el peor caso, aunque usa memoria temporal.

Quick Sort usa un pivote. En promedio es rapido, pero puede caer en $O(n^2)$ si los pivotes producen divisiones desbalanceadas. Shell Sort, el algoritmo extra de mi Actividad 1.5, compara elementos separados por espacios que disminuyen gradualmente. Esto reduce movimientos antes de la pasada final.

Interpretacion de mediciones

Con 6818 registros, Selection Sort tardo 802,415 microsegundos en log607-1. Merge Sort tardo 75,323 microsegundos en el mismo archivo. La diferencia observada concuerda con $O(n^2)$ frente a $O(n \log n)$. Shell Sort obtuvo 14,661, 25,902 y 24,769 microsegundos en distintas corridas de log607-2. La variacion entre corridas puede deberse a otros procesos del equipo, pero en todos los casos fue rapido con los datos casi ordenados.

Antes de ejecutar Shell Sort predije que seria rapido porque log607-2 ya estaba casi ordenado y esperaba pocos movimientos. La primera y la tercera medicion coincidieron con esa prediccion. Para Merge Sort tambien predije un resultado rapido debido a su complejidad $O(n \log n)$, y la medicion fue consistente con esa expectativa.

Uso critico de inteligencia artificial

Utilice IA para aclarar sintaxis de C++, adaptar los algoritmos de `vector<int>` a `vector<Log>` y revisar partes especificas del programa. Algunos prompts fueron: 'dame un commit basico para comparar fechas', 'explicame el codigo hasta el momento' y 'como manejo timestamps duplicados con busqueda binaria'. Revise cada sugerencia, la compile y compare la salida con los archivos originales.

Un error concreto aparecio en una sugerencia inicial de lectura: se asumio que el archivo no contenia el año. Como consecuencia, 2025 se guardaba como hora, la hora se guardaba como IP y el ordenamiento solo parecia correcto por mes y día. Detecte el problema al observar una salida como 'Jan 1 2025 08:24:58...'. Corregi el parser para leer month, day, year, time e ip por separado.

Tambien revise un punto algoritmico en la busqueda por rango. Usar una sola busqueda binaria para una coincidencia exacta no garantiza encontrar el primero ni el ultimo de varios timestamps iguales. Esa estrategia podia perder duplicados. La correccion fue implementar `buscarInicio` como limite inferior y `buscarFin` como limite superior. El ciclo usa el intervalo [inicio, fin), por lo que cada registro duplicado aparece una vez.

La IA sirvio como apoyo, pero no sustituyo la revision. Probar el programa con salidas reales fue necesario para encontrar el error de lectura y verificar los limites de la busqueda.

Conclusion personal

Esta actividad me permitio relacionar la complejidad teorica con tiempos reales. Los algoritmos cuadraticos son comprensibles, pero pierden utilidad conforme aumenta el numero de registros. Merge Sort ofrece un rendimiento estable, mientras que Insertion y Shell Sort pueden aprovechar cierto orden previo. Tambien aprendi que una busqueda binaria correcta no solo encuentra un elemento: debe definir claramente los limites, especialmente cuando existen valores duplicados.